



PES

UNIVERSITY

CELEBRATING 50 YEARS

ALGORITHMS ANALYSIS AND DESIGN

Lekha A

Computer Applications

ALGORITHMS ANALYSIS AND DESIGN

Introduction, Analysis Framework and Sorting Techniques

Sorting Algorithms – Brute Force Paradigm

Lekha A

Computer Applications



ALGORITHMS ANALYSIS AND DESIGN

Stable sorting algorithm

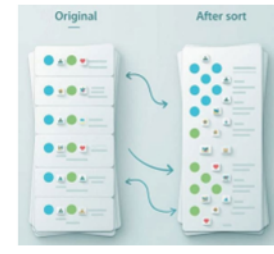


- A stable sorting algorithm is said to be one that preserves the order of elements that have the same key.
- If two elements with the same key are in a particular order in the original list, they will remain in the same order in the sorted list.



ALGORITHMS ANALYSIS AND DESIGN

Stable sorting algorithm

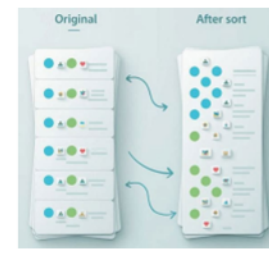


- Example
 - Take a list of students who are first sorted by their names and then by their grades.
 - If a stable sorting algorithm is used on this list, the students with the same name will be in the same order after the sorting process, no matter what changes are made to their grades.



ALGORITHMS ANALYSIS AND DESIGN

Stable sorting algorithm



- Examples
 - Merge Sort
 - Insertion Sort



ALGORITHMS ANALYSIS AND DESIGN

In-place sorting algorithm



- This is a sorting algorithm that sorts the input array in place.
 - It does not require additional memory to store the sorted elements.
- They are commonly used in situations where memory is limited.
- They are widely used to sort lists of elements in the memory of a computer.



ALGORITHMS ANALYSIS AND DESIGN

In-place sorting algorithm



- Examples
 - Bubble Sort
 - Selection Sort
 - Quick Sort
 - Shell Sort
 - Insertion Sort



ALGORITHMS ANALYSIS AND DESIGN

Stable vs. In-place sorting algorithms



Criteria

Stable Sorting Algorithm

In-Place Sorting Algorithm

Definition

Preserves the relative order of elements with equal keys after sorting.

Sorts elements within the same array using constant or minimal extra space.

Key Property

Stability is guaranteed for equal elements.

Memory efficiency is prioritized. Stability may not be guaranteed.

Space Usage

May require additional memory depending on implementation.

Uses $O(1)$ extra space in most cases.



ALGORITHMS ANALYSIS AND DESIGN

Stable vs. In-place sorting algorithms



Criteria	Stable Sorting Algorithm	In-Place Sorting Algorithm
When Required	Required when multiple attributes are sorted sequentially and order of equal keys must be preserved.	Used when memory usage must be minimized and stability is not a concern.
Common Algorithms	Insertion Sort, Merge Sort.	Bubble Sort, Selection Sort.



ALGORITHMS ANALYSIS AND DESIGN

Stable vs. In-place sorting algorithms

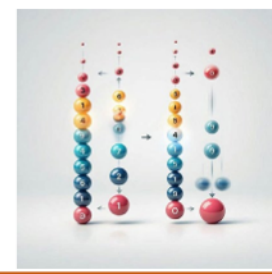


Criteria	Stable Sorting Algorithm	In-Place Sorting Algorithm
Example Scenario	Sorting students by name and then by grade while preserving the original grade order for identical names.	Sorting a list of numbers where the order of equal values does not matter.
Selection Criteria	Choose when relative ordering of equal elements affects correctness.	Choose when space efficiency is more important than preserving order.



ALGORITHMS ANALYSIS AND DESIGN

Bubble Sort

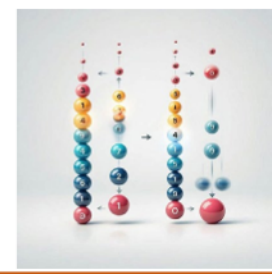


- It involves repeatedly going through the list of items, comparing them and swapping them if they are in the wrong order.
- The process of going through the list is continued until no swaps are required.
- It is named for the reason that the smaller items "bubble" to the top of the list with each pass, while the larger items sink to the bottom.



ALGORITHMS ANALYSIS AND DESIGN

Bubble Sort - Steps

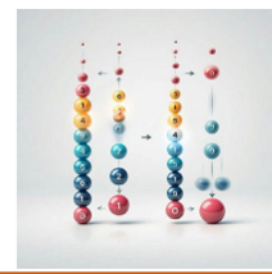


- Begin by comparing the first two elements of the array.
- If the first element is larger than the second, swap them.
- Proceed to the next pair of elements and compare them.
- Continue this process until the entire array has been compared.



ALGORITHMS ANALYSIS AND DESIGN

Bubble Sort - Steps

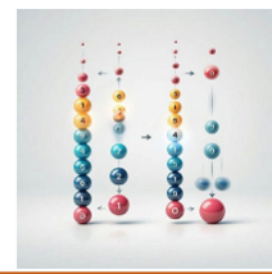


- This is one complete pass and will place the largest element at the end of the array.
- This process will be repeated for the entire array but without comparing the last elements that have been sorted.



ALGORITHMS ANALYSIS AND DESIGN

Algorithm - BubbleSort($A[0...n-1]$)

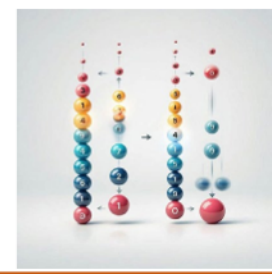


- //Sorts the array using bubble sort.
- //Input: An array $A[0...n-1]$ of orderable elements.
- //Output: An Array $A[0...n-1]$ in ascending order.



ALGORITHMS ANALYSIS AND DESIGN

Algorithm - BubbleSort($A[0...n-1]$)

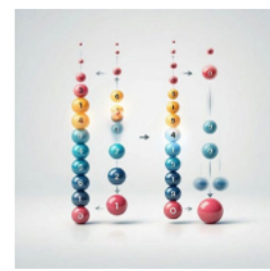


- for $i \leftarrow 0$ to $n-2$ do
- for $j \leftarrow 0$ to $n-2-i$ do
- if $A[j+1] < A[j]$
- swap $A[j+1]$ and $A[j]$



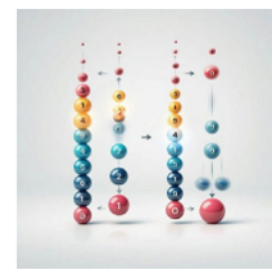
ALGORITHMS ANALYSIS AND DESIGN

Algorithm - BubbleSort(A[0...n-1]) - Tracing



- $A = [64, 34, 25, 12, 22, 11, 90]$

Pass No.	i	$i \leq n-2$	j	$j \leq n-2-i$	$A[j+1] < A[j]$	Task	A						
							0	1	2	3	4	5	6
1	0	$0 \leq 5$ (T)	0	$0 \leq 5$ (T)	$34 < 64$ (T)	swap 64 and 34	34	64	25	12	22	11	90
			1	$1 \leq 5$ (T)	$25 < 64$ (T)	swap 25 and 64	34	25	64	12	22	11	90
			2	$2 \leq 5$ (T)	$12 < 64$ (T)	swap 12 and 64	34	25	12	64	22	11	90
			3	$3 \leq 5$ (T)	$22 < 64$ (T)	swap 22 and 64	34	25	12	22	64	11	90
			4	$4 \leq 5$ (T)	$11 < 64$ (T)	swap 11 and 64	34	25	12	22	11	64	90
			5	$5 \leq 5$ (T)	$90 < 64$ (F)								
2	1	$1 \leq 5$ (T)	6	$6 \leq 5$ (F)			34	25	12	22	11	64	90
			0	$0 \leq 4$ (T)	$25 < 34$ (T)	swap 25 and 34	25	34	12	22	11	64	90
			1	$1 \leq 4$ (T)	$12 < 34$ (T)	swap 12 and 34	25	12	34	22	11	64	90
			2	$2 \leq 4$ (T)	$22 < 34$ (T)	swap 22 and 34	25	12	22	34	11	64	90
			3	$3 \leq 4$ (T)	$11 < 34$ (T)	swap 11 and 34	25	12	22	11	34	64	90
			4	$4 \leq 4$ (T)	$64 < 34$ (F)								
			5	$5 \leq 4$ (F)			25	12	22	11	34	64	90



ALGORITHMS ANALYSIS AND DESIGN

Algorithm - BubbleSort(A[0...n-1]) - Tracing

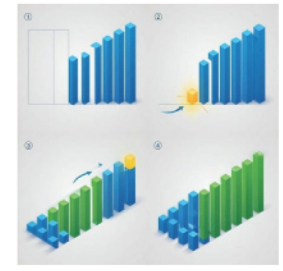
- A=[64, 34, 25, 12, 22, 11, 90]

Pass No.	i	$i \leq n-2$	j	$j \leq n-2-i$	$A[j+1] < A[j]$	Task	A						
3	2	$2 \leq 5$ (T)	0	$0 \leq 3$ (T)	$12 < 25$ (T)	swap 12 and 25	12	25	22	11	34	64	90
			1	$1 \leq 3$ (T)	$22 < 25$ (T)	swap 22 and 25	12	22	25	11	34	64	90
			2	$2 \leq 3$ (T)	$11 < 25$ (T)	swap 11 and 25	12	22	11	25	34	64	90
			3	$3 \leq 3$ (T)	$34 < 25$ (F)								
			4	$4 \leq 3$ (F)			12	22	11	25	34	64	90
4	3	$3 \leq 5$ (T)	0	$0 \leq 2$ (T)	$22 < 12$ (F)								
			1	$1 \leq 2$ (T)	$11 < 22$ (T)	swap 11 and 22	12	11	22	25	34	64	90
			2	$2 \leq 2$ (T)	$25 < 22$ (F)								
			3	$3 \leq 2$ (F)			12	11	22	25	34	64	90
5	4	$4 \leq 5$ (T)	0	$0 \leq 1$ (T)	$11 < 12$ (T)		11	12	22	25	34	64	90
			1	$1 \leq 1$ (T)	$22 < 12$ (F)								
			2	$2 \leq 1$ (T)			11	12	22	25	34	64	90
6	5	$5 \leq 5$ (T)	0	$0 \leq 0$ (T)	$12 < 11$ (F)								
				$1 \leq 0$ (F)			11	12	22	25	34	64	90



ALGORITHMS ANALYSIS AND DESIGN

Selection Sort

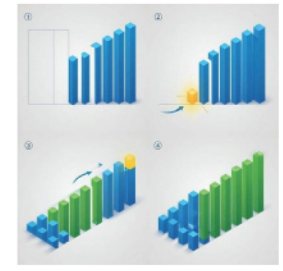


- The algorithm splits the input list into two segments: the sorted part and the unsorted part.
- At the start, the sorted part is empty, and the unsorted part holds all the elements.



ALGORITHMS ANALYSIS AND DESIGN

Selection Sort

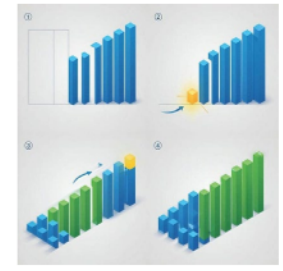


- The algorithm keeps adding the smallest element from the unsorted part to the end of the sorted part.
- This goes on until the unsorted part is empty, and the sorted part holds all the elements in the correct order.



ALGORITHMS ANALYSIS AND DESIGN

Algorithm - SelectionSort($A[0\dots n-1]$)

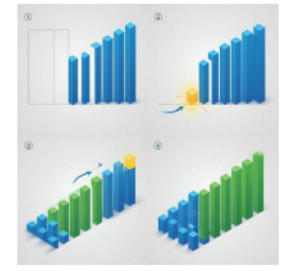


- // Sorts given array using selection sort.
- //Input: An array $A[0\dots n-1]$ orderable elements.
- //Output: An array $A[0\dots n-1]$ sorted in ascending order.



ALGORITHMS ANALYSIS AND DESIGN

Algorithm - SelectionSort($A[0 \dots n-1]$)

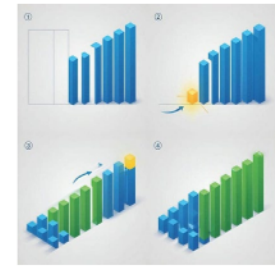


- for $i \leftarrow 0$ to $n-2$ do
 - $\text{min} \leftarrow i$
 - for $j \leftarrow i+1$ to $n-1$ do
 - if $A[j] < A[\text{min}]$
 - $\text{min} \leftarrow j$
 - swap $A[i]$ and $A[\text{min}]$



ALGORITHMS ANALYSIS AND DESIGN

Algorithm - SelectionSort(A[0....n-1]) - Tracing



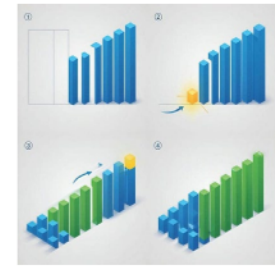
- $A = [64, 34, 25, 12, 22, 11, 90]$

Pass No.	i	$i \leq n-2$	min	j	$j \leq n-1$	$A[j] < A[\text{min}]$	A						
							0	1	2	3	4	5	6
							64	34	25	12	22	11	90
1	0	$0 \leq 5$ (T)	0	1	$1 \leq 6$ (T)	$34 < 64$ (T)							
			1	2	$2 \leq 6$ (T)	$25 < 34$ (T)							
			2	3	$3 \leq 6$ (T)	$12 < 25$ (T)							
			3	4	$4 \leq 6$ (T)	$22 < 12$ (F)							
				5	$5 \leq 6$ (T)	$11 < 12$ (T)							
			4	6	$6 \leq 6$ (T)	$90 < 11$ (F)							
				7	$7 \leq 6$ (F)		11	34	25	12	22	64	90
2	1	$1 \leq 5$ (T)	1	2	$2 \leq 6$ (T)	$25 < 34$ (T)							
			2	3	$3 \leq 6$ (T)	$12 < 25$ (T)							
			3	4	$4 \leq 6$ (T)	$22 < 12$ (F)							
				5	$5 \leq 6$ (T)	$64 < 12$ (F)							
				6	$6 \leq 6$ (T)	$90 < 12$ (F)							
				7	$7 \leq 6$ (F)		11	12	25	34	22	64	90



ALGORITHMS ANALYSIS AND DESIGN

Algorithm - SelectionSort(A[0....n-1]) - Tracing



- A=[64, 34, 25, 12, 22, 11, 90]

Pass No.	i	i≤n-2	min	j	j≤n-1	A[j]<A[min]	A						
3	2	2≤5 (T)	2	3	3≤6 (T)	34<25 (F)							
				4	4≤6 (T)	22<34 (T)							
			4	5	5≤6 (T)	64<22 (F)							
				6	6≤6 (T)	90<64 (F)							
				7	7≤6 (F)		11	12	22	34	25	64	90
4	3	3≤5 (T)	3	4	4≤6 (T)	25<34 (T)							
			4	5	5≤6 (T)	64<25 (F)							
				6	6≤6 (T)	90<25 (F)							
				7	7≤6 (F)		11	12	22	25	34	64	90
5	4	4≤5 (T)	4	5	5≤6 (T)	64<34 (F)							
				6	6≤6 (T)	90<34 (F)							
				7	7≤6 (F)		11	12	22	25	34	64	90
6	5	5≤5 (T)	5	6	6≤6 (T)	90<64 (F)							
				7	7≤6 (F)		11	12	22	25	34	64	90



ALGORITHMS ANALYSIS AND DESIGN

Recap

- Types
 - Stable
 - In-Place
- Bubble Sort
- Selection Sort



THANK YOU

Lekha A
Computer Applications